

Weather API Proxy

1. Project Title

Weather API Proxy

2. Project Overview

A Weather API Proxy is a backend service that acts as an intermediary between users and a weather data provider. Instead of calling a weather API directly, users send requests to your backend, which fetches weather data from a third-party weather service and returns a simplified response.

Example Flow

Plain text

User Request

↓

Weather API Proxy

↓

Weather Service API

↓

Weather API Proxy

↓

User Response

The proxy helps secure API keys, improve performance through caching, and customize responses.

3. Problem Statement

Most weather providers require:

API keys

Complex requests

Paid usage limits

Direct access can expose API keys and increase unnecessary API calls.

A Weather API Proxy solves these issues by:

Hiding API keys

Reducing external requests

Simplifying weather data

4. Objectives

Fetch weather information from external APIs.

Hide third-party API keys.

Provide simplified weather responses.

Cache weather data for faster performance.

Handle API errors efficiently.

5. Scope of the Project

The system allows users to:

Weather Information

Current Weather

Temperature

Humidity

Wind Speed

Atmospheric Pressure

Forecast Services

Daily Forecast

Weekly Forecast

Weather Alerts

Location Search

Search by City Name

Search by Coordinates

6. System Architecture

Plain text

Client Application

|



Weather API Proxy Server

|



Weather Provider API

|



Database / Cache

Components

Client

Sends weather requests.

Proxy Server

Receives requests.

Calls external weather API.

Processes data.

Weather Provider

Examples:

OpenWeather

WeatherAPI

Cache Storage

Stores recent weather data.

Reduces API requests.

7. Functional Requirements

Current Weather Module

Users can request current weather.

Input:

City Name

Output:

Temperature

Humidity

Wind Speed

Example:

Plain text

City: Chennai

Response:

JSON

```
{  
  "city": "Chennai",  
  "temperature": "34°C",  
  "humidity": "65%",  
  "windSpeed": "15 km/h"  
}
```

Forecast Module

Users can view future weather forecasts.

Input:

City Name

Output:

5-Day Forecast

Daily Temperatures

Search Module

Search weather by:

City Name

Example:

Plain text

Madurai

Geographic Coordinates

Example:

Plain text

Latitude: 9.9252

Longitude: 78.1198

Error Handling Module

Handle errors such as:

Invalid city names

API failures

Network issues

Example:

JSON

```
{  
  "error": "City not found"  
}
```

8. Non-Functional Requirements

Security

Protect API keys

Validate user input

Performance

Fast response times

Reliability

Accurate weather information

Scalability

Handle many requests simultaneously

9. Database Design (Optional)

Weather Cache Table

Field

Type

id

Integer

city

Varchar

temperature

Float

humidity

Integer

wind_speed

Float

fetches_at

DateTime

Purpose:

Store recent weather responses.

Reduce external API calls.

10. API Endpoints

Get Current Weather

Http

GET /api/weather?city=Chennai

Get Forecast

Http

GET /api/forecast?city=Chennai

Get Weather by Coordinates

Http

GET /api/weather?lat=9.9252&lon=78.1198

Health Check

Http

GET /api/health

Checks if the server is running.

11. Technology Stack

Backend

Node.js

Framework

Express.js

API Client

Axios

Database (Optional)

MongoDB

Cache

Redis

Testing Tool

Postman

12. Project Workflow

Step 1

User sends a weather request.

Example:

Http

GET /api/weather?city=Chennai

Step 2

Server validates the city name.

Step 3

Server checks cache.

Step 4

If data exists in cache:

Return cached data.

Step 5

If cache is empty:

Request data from weather provider.

Step 6

Receive weather information.

Step 7

Store response in cache.

Step 8

Return formatted weather data to the user.

13. Sample API Request

Request

Http

GET /api/weather?city=Madurai

Response

JSON

```
{  
  "city": "Madurai",  
  "temperature": "33°C",  
  "humidity": "58%",  
  "windSpeed": "12 km/h",  
  "condition": "Partly Cloudy"  
}
```

14. Advantages

Protects API keys

Faster responses through caching

Simplified weather data

Reduced API costs

Easy integration with web and mobile apps

15. Future Enhancements

User Authentication

Allow registered users to save favorite cities.

Weather Notifications

Send alerts for:

Rain

Storms

Extreme temperatures

Historical Weather Data

View previous weather records.

Weather Dashboard

Display:

Charts

Forecast trends

Temperature comparisons

Multi-Provider Support

Fetch data from multiple weather services and choose the most reliable result.

16. Project Conclusion

The Weather API Proxy is a backend application that retrieves weather information from external weather services and delivers it securely and efficiently to users. It demonstrates important backend concepts such as API integration, caching, request handling, data processing, error management, and security. This makes it an excellent project for learning real-world backend development and showcasing API integration skills in academic and professional portfolios.

Current & 24-hr



Weather Forecast

Sunday

April 17th
Toronto



7°C

Heavy Cloud

PREDICTABILITY 71 %
HUMIDITY 41 %
WIND 9 km/h
AIR PRESSURE 1025.5 mb
MAX TEMP 7 °C
MIN TEMP -1 °C



Mon

6°



Tue

5°



Wed

9°



Thu

16°



Fri

12°

✓ JS Weather API __PLACE_UNDER_ORTHO_CAMERA



✓ Multiple Day Forecast [TRY ME]



✓ WeatherBg

21 17 1

> Weather - Day 1



> Weather - Day 2



> Weather - Day 3



> Weather - Day 4



> Weather - Day 5



loading

21 17 17